

TRANSFORMERS FROM SCRATCH

REPORT

Ashmit Chamoli

Introduction

This repository contains an implementation of the Transformer architecture, introduced in the paper “Attention is All You Need” by Vaswani et al. in 2017. The implementation uses basic PyTorch classes such as `Linear` and `LayerNorm`.

Questions

Question 1

What is the purpose of self-attention, and how does it facilitate capturing dependencies in sequences?

Answer

Self-attention is a core mechanism in the Transformer architecture that allows models to evaluate the importance of different elements within an input sequence relative to one another. This capability is crucial for capturing dependencies and relationships in sequences.

Purpose of Self-Attention. The main purpose of self-attention is to enable a model to focus on various parts of an input sequence while processing it, thereby enhancing its understanding of context and relationships among the elements. Unlike RNNs, which process sequences sequentially and struggle with long-range dependencies, self-attention allows for parallel processing of all elements in a sequence. This serves many purposes:

- **Capturing Long-Range Dependencies:** Self-attention can relate distant elements within a sequence effectively. For instance, in a sentence, the meaning of a word may depend on another word located far away. Self-attention allows the model to consider all words simultaneously, making it easier to capture such dependencies.
- **Contextual Understanding:** Each element in the sequence can attend to all other elements, allowing the model to build a more nuanced understanding of context. This is particularly important in tasks like sentiment analysis or machine translation, where the meaning can change based on surrounding words.
- **Dynamic Weighting:** Self-attention computes attention scores that determine how much focus each element should have on others. This dynamic approach allows the model to prioritize relevant information based on the specific task it is being trained for.
- **Efficiency:** The ability to compute self-attention in parallel makes it computationally efficient compared to sequential models. This efficiency is particularly beneficial when dealing with large datasets or long sequences.

Mechanism of Self-Attention. First, 3 components are derived from the input embeddings: Queries (Q), Keys (K), and Values (V). Second, attention scores are computed by taking the dot product of Q and K, followed by scaling and applying a softmax function to obtain normalized weights. Third, these weights are then used to compute a weighted sum of the V vectors, resulting in an output that **reflects contextual information from across the entire sequence**. To capture different types of relationships, multiple sets of Q, K, and V are created (multi-head attention), allowing the model to learn various aspects of context simultaneously.

How it handles dependencies in Machine Translation. In machine translation, self-attention enables the model to understand and generate translations that maintain contextual integrity across languages. For example, when translating a sentence from English to French, self-attention allows the model to focus on relevant words in both languages simultaneously. It can weigh how much attention each word should receive based on its relationship with others in the sentence. This capability helps ensure that nuances such as idiomatic expressions or grammatical structures are preserved during translation, leading to more accurate and fluent outputs.

Question 2

Why do transformers use positional encodings in addition to word embeddings? Explain how positional encodings are incorporated into the transformer architecture. Briefly describe recent advances in various types of positional encodings used for transformers and how they differ from traditional sinusoidal positional encodings.

Answer

Transformers use positional encodings in addition to word embeddings to provide information about the order of words in a sequence. Transformers process all input tokens simultaneously. This parallel processing means that the model does not have any built-in understanding of the sequence's structure or the relative positions of tokens. Reordering the sequence will simply swap the corresponding rows and columns in the attention matrix. Positional encodings help bridge this gap by explicitly incorporating information about the position of each token in the input sequence. **Purpose of Positional Encodings.**

- **Sequential Information:** The order of words is crucial for understanding meaning. For instance, "The boy drank cola." has a different meaning than "The cola drank boy." Positional encodings allow the model to differentiate between these sequences by providing unique representations for each position.
- **Combining with Word Embeddings:** Positional encodings are added to word embeddings before they are fed into the Transformer model. This addition ensures that each token's representation includes both its semantic meaning (from the word embedding) and its position in the sequence (from the positional encoding).
- **Facilitating Attention Mechanism:** The self-attention mechanism in Transformers relies on understanding relationships between tokens based on their positions. By incorporating positional encodings, the model can effectively weigh how much attention to give to different tokens based on their respective positions.

Incorporation into Transformer Architecture The positional encodings are computed using sine and cosine functions with varying frequencies. The mathematical formulation is as follows: For even indices:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

For odd indices:

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Here pos is the position of the token in the sequence. i refers to the dimension within the positional encoding vector. d_{model} is the dimension of the input embeddings. These positional encoding vectors are then added element-wise to the corresponding word embeddings, resulting in a combined representation that retains both semantic and positional information.

Recent Advances in Positional Encodings

- **Learned Positional Embeddings:** Instead of using fixed sinusoidal functions, some models utilize learned embeddings for each position in a sequence. This approach allows the model to learn optimal positional representations during training, potentially improving performance on specific tasks.
- **Relative Positional Encodings:** These encodings focus on capturing relative distances between tokens rather than absolute positions. This method can be particularly beneficial for tasks where the relationship between tokens is more important than their specific locations within a sequence.
- **Attention-Based Positional Encoding:** Some recent architectures integrate positional information directly into the attention mechanism itself, allowing for dynamic adjustments based on context rather than relying solely on pre-defined encodings.

- **Rotary Positional Encodings:** This method modifies how positional information is represented by rotating token embeddings in a high-dimensional space based on their positions. This approach has shown promise in enhancing model performance while maintaining efficiency.

These advancements differ from traditional sinusoidal positional encodings primarily in their adaptability and focus on relative positioning, which can lead to improved performance across various tasks by better capturing contextual relationships within sequences.

References:

- <https://aiml.com/explain-the-need-for-positional-encoding-in-transformer-models/>
- <https://towardsdatascience.com/attention-and-transformer-models-fe667f958378?gi=f1cdace1cfe5>
- <https://machinelearningmastery.com/a-gentle-introduction-to-positional-encoding-in-transformer-models-part-1/>

Hyperparameter Tuning

In this section we will see the effect of hyperparameters on model performance and training in two-folds.

- First we will see the effect of the number of encoder/decoder stacks on model training.
- Secondly, we will observe how parameters like, embedding dimension, number of attention heads and dropout etc. effect the model performance.

Effect of Number of Layers on Training

First let us see how the model performance changes as number of layers is varied.

We use embedding size = 64, dim-feedforward = 128, activation = relu, dropout = 0.1 and nHeads = 1. These hyperparameters are kept fixed so that the difference in performance can be attributed to the number of layers alone.

Num Layers	Bleu Score (avg.)	Accuracy
1	0.457	0.729
3	0.481	0.750

We can see that the performance is similar for both the models. The improvement gained from adding 2 extra layers is not substantial.

Now let us look at some training plots:

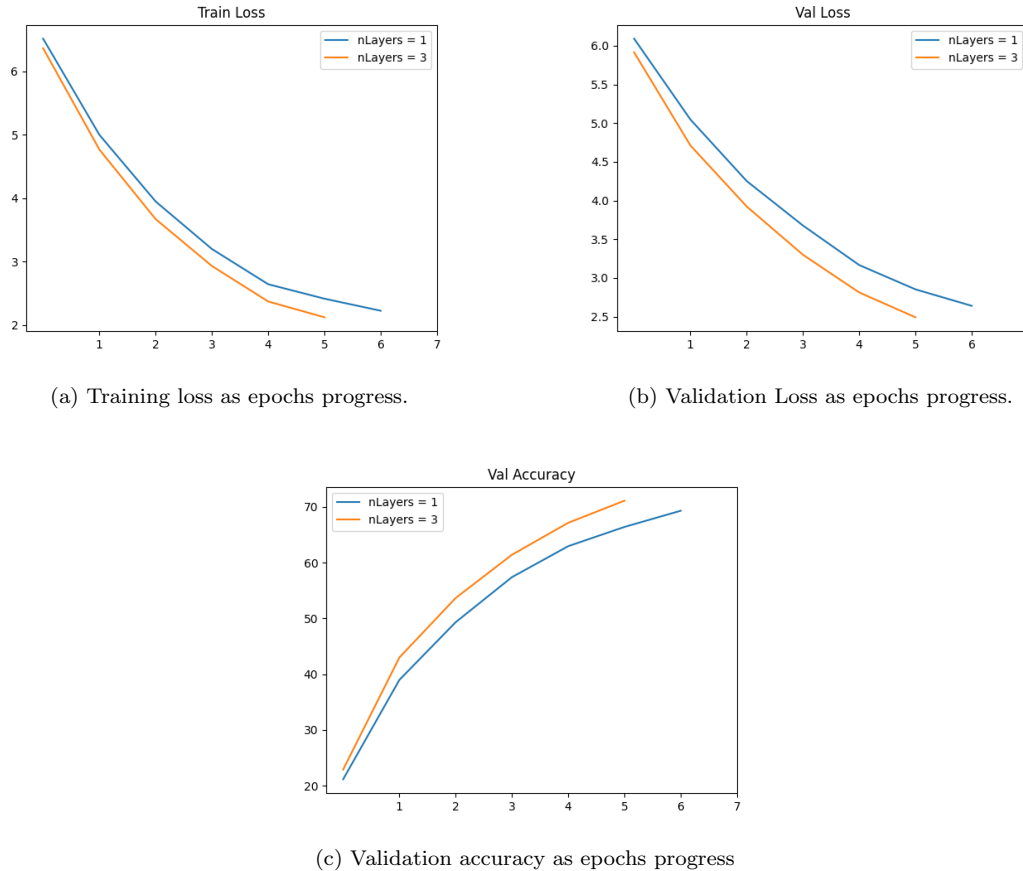


Figure 1

We see that the model with 3 layers converges faster than the model with 1 layer. However, both models converge to the same loss eventually and achieve similar performance. This is because the purpose of having multiple encoder-decoder layers is to accelerate the crystallization of useful information. The relevant information and interactions snowball from one layer to the next. This process is more difficult as the number of layers decreases as each layer adds to the complexity of interactions among the tokens.

Effect of Other Hyperparameters

Now we will see the effect of parameters such as nHeads,

Effect of number of attention heads. Here we will keep embedding size = 512, nLayers = 6, dimFeedforward = 1024. We will vary nHeads and compare performance.

Num Heads	Bleu Score (avg.)	Accuracy
1	0.818	0.932
8	0.826	0.935

As the number of attention heads increase, the performance improves substantially. This is because each head learns a different aspect of the embedding/task. It is somewhat like having an ensemble of models and combining the results, thus boosting performance and stability of output.

In this case, we see very slight improvements which might not be significant. But for more complex tasks and datasets we will see a wider gap.

Effect of embedding size. Here we will keep nHeads = 1, nLayers = 1, dimFeedforward = 2*(embedding size). We will vary embedding size and compare performance.

Embedding Dim.	Bleu Score (avg.)	Accuracy
64	0.457	0.729
512	0.863	0.950

Embedding size has a significant effect on performance. More information can be extracted in a bigger embedding leading to improved performance.

Overall, we get best performance for embedding dim. = 256, nHeads = 8, nLayers = 6, dim-feedforward = 1024, dropout = 0.1.